

The subject of embedded system development is quite vast. It is not fair to expect both breadth and depth from a fresh recruit. At the undergraduate level, we expect a good understanding of fundamental concepts of electronics and exposure to various design modules (both digital and analogue) from practical point of view.



—K.S. Mittal, additional general manager, ECIL

20,000 for degree holders. Whereas, the private sector is always ready to offer more lofty figures for candidates with suitable exposure. The ability to multitask may be a defining factor.

You could begin with a purely technical role and continue as per your aptitude within the chip-to-ship chain of the embedded system. A strong technical foothold along with a desire for innovation can lead you to senior positions in research and development. Nonetheless, it is always advisable to explore the business processes as that can accelerate the climb to the top.

Some suggestions

Though embedded system itself is a specialisation, the industry expects you to have super specialisations within the field like embedded 'C' programming, exploring controller-specific features under embedded 'C' RTOS porting and application development, and special networking protocols along with the fundamentals. In addition, there are certain other skills which are categorised as 'must-haves.' Knowledge of 'C' programming and operating system concepts, and exposure to microcontrollers and basic electronics are typical examples.

In addition to the basic electronics that I discussed in the microcontrollers article last month, knowledge of KEIL compiler and designing concept (both

analogue and digital) would be an advantage for any aspiring candidate.

From my interaction with different industry spokespersons, I inferred that communication gap among team members could create a roadblock in any given project. A lot of it has to do with the specific jargon used by the different groups such as the software engineers and the hardware engineers; both also find it difficult to communicate with application domain experts. To avoid this hazard, your employer may want to judge your overall knowledge base and communication skills at the interview table. The necessary foundations of both the hardware building blocks and the software tools are equally important to learn as you embark on your journey.

Additionally, try to understand the basics of any one of the representative application domains of your choice so that, in future, you can take up the challenge of embedded system development in any application area.

How to begin?

The best way of starting work on embedded systems is to work on the evaluation boards of popular microcontrollers or processors. Understand the system. Then try to write the software that works on these boards. In due course you can reveal the architecture of the system as well as the uti-

lisation of software tools. Even if you do not have access to an evaluation board, you can get preliminary idea of embedded software development by experimenting with the code on any desktop computer. After gaining the necessary knowledge of the hardware to write the firmware, you will be in a position to be a member of any embedded system project team.

The first and foremost thing you need to know is the categorisation of embedded systems based on their functionality. All embedded systems may be bucketed under some broad categories like standalone system, real-time systems, network information appliances and mobile devices. Each of them is unique with respect to inputs, outputs and custom-built hardware architecture. Some exposure to different architectures (advantages and philosophy) and their comparative study (RISC vs CISC) are must. Try to deal with the building blocks of the hardware like processors, RAMs, ROMs, clock circuitry to generate clock signal, input/output devices, sensors and transducers, ADCs and DACs, DMA controllers, timers and power supply units.

Now you have to make the embedded system communicate with the external world. Always keep in mind that embedded software development involves cross-platform development as the software is initially developed on a host system and then the executable image is transferred to the target system. The know-how of board details and specifications and other hardware tools (digital multimeters, logic analysers and oscilloscopes), system clock and fuse bits, Linux and various distributions, operating system development suite (API calls), cross-platform development tools (overall GNU Project), Free Software Foundation (FSF) philosophy, ROM emulator, EPROM programmer, etc may be turnkey for this purpose.

Learn now to make the system work. At this point of execution, you have to write a program for the embed-